AXEL SPRINGER TECH QA MANAGER CHALLENGE · IOS & ANDROID PLATFORMS

Screenshot Testing Strategy for Mobile News Apps

Ensuring visual stability and UI determinism at scale for millions of BILD & WELT readers.

PREPARED BY
Ilona Zakharava

CORE TOOLCHAIN
Paparazzi (Android) & Swift-Snapshot (iOS)

CI/CD SPEED
Off-device JVM rendering · full suite ~3 min on CI incl. build

Visual Testing Strategy Goals

Fast Time-to-Market

Reduce visual regression runs from days to seconds on every Pull Request via off-device bytecode rendering.

Determinism

Eliminate flakiness by decoupling from backends and live networks using deterministic mock fixtures and static clock times.

Edge Case Coverage

Validate long German compound headlines, Dark Mode, 200% Dynamic Type, Right-to-Left (RTL), and Paywall premium states.

News App Test Pyramid

E2E Tests (5–10%)
Critical user journeys (Subscriptions, Auth, In-app purchase flows)

Integration / Snapshot Tests (20–30%)
UI Components + Mock Fixtures (Paparazzi / Swift-Snapshot)

Unit Tests (60–70%)
Business logic, JSON parsing, Paywall rules, relative date formatting

Why Snapshot = Integration?

- **Isolation:** Layout tests do not depend on backend availability, network latency, or database states.
- **Speed:** Executed on the JVM / native process without heavy emulator or simulator boots.
- **Reproducibility:** Same code, same pixels: re-recording on CI produced byte-identical goldens.

02B. UNIT TEST SUITE

Mandatory Unit Tests Checklist (Android & iOS)

Pure Logic & Formatting

Read Time Calculation: Test `calculateReadTime(text)` for word count boundary limits.

German Relative Date: Validate timestamp conversion to German strings (e.g., *"Vor 5 Minuten"*).

Null-Safety JSON Parsing: Validate response deserialization when optional fields are missing.

Business & Paywall Rules

Paywall Access Guard: Verify `shouldShowPaywall(user, article)` for free vs. WELT+ / BILD Plus tier.

Category Filter Logic: Ensure filter algorithms isolate articles matching *"Politik"* or *"Sport"* tags.

Offline Storage Cache: Check SQLite/Room/CoreData persistence & retrieval of bookmarked articles.

⚡ Fast execution on JUnit / XCTest

Business rules verified before UI renders

Target Device Profile (Germany Region)

iOS Ecosystem

- **OS Versions:** iOS 16, 17, 18
- **Smartphones:** iPhone 13 / 14 / 15 / 16, iPhone SE
- **Tablets (High ARPU):** iPad 10.9", iPad Pro 13"
- **Features:** Dynamic Type (up to 200%), Dark Mode support

Android Ecosystem

- **OS Versions:** Android 12–15 (Min SDK 24)
- **Smartphones:** Samsung Galaxy S23 / S24, A54 / A55, Pixel 8
- **Tablets:** Samsung Galaxy Tab A9+, Tab S9
- **Features:** Diverse aspect ratios (FHD+), system font scaling

Tablet Testing Strategy & Mandatory Checklist

Tablet Popularity & ARPU Impact

Tablets are a key reading device for digital newspaper subscribers (to be confirmed with internal analytics).

- **Top iOS:** iPad 10.9" (Base/Air), iPad Pro 13" (~50%+ share)
- **Top Android:** Samsung Galaxy Tab A9+, Tab S9

1. Multi-Column Grid Balancing
Verify 2-column & 3-column feed layouts populate without orphaned whitespace.

2. Master-Detail (Split View)
Ensure left feed list and right article detail stay synchronized on iPadOS.

3. Orientation Switch (Portrait / Landscape)
Snapshot verification when rotating screen dimensions.

4. High-DPI Image Scaling
Validate high-resolution media asset loading on iPad Retina displays.

04B. TABLET TOOLING MATRIX

Tablet Snapshot Tooling Evaluation & Final Selection

Tool / Approach	Tablet Simulation Method	Multi-Column Rendering	CI Speed & Overhead	Final Selection Verdict
Paparazzi (Android Tablets)	DeviceConfig.PIXEL_TABLET	Wide-window layout rendered off-device	Fast / Linux runner	WINNER (Android Tablet)
Swift-Snapshot (iPadOS)	Native `.image(on: .iPadPro12_9)` trait	Native SwiftUI / SplitView	Fast / macOS runner	WINNER (iPadOS)
Appium Tablet Grid	Physical / Cloud Tablet Emulators	Full App UI Automation	Very Slow (5+ mins) / Expensive	REJECTED: Flaky for layout diffs
BrowserStack / Sauce Labs	Real Physical iPad & Tab Devices	Real Hardware Render	Moderate / High SaaS Cost	REJECTED: Reserved for Nightly E2E

> **Conclusion for Tablets:** Paparazzi with tablet device configs (Android) and Swift-Snapshot-Testing with iPad view traits (iOS) give deterministic tablet snapshots without paying for cloud device grids on every PR. Multi-column layouts are tested once the product has them; JetNews uses a single-column wide-window feed.

04C. PLATFORM ARCHITECTURE MATRIX

Android (Paparazzi) vs. iOS (Swift-Snapshot-Testing)

Technical Dimension	Android (Paparazzi)	iOS (Swift-Snapshot-Testing)
Core Technology & Origin	Cash App / Block	Point-Free
Execution Runtime	Host JVM via Android Studio Layoutlib	Headless iOS Simulator Process
Emulator Dependency	NO (off-device JUnit execution)	YES (Virtual off-screen simulator required)
Execution Speed	Seconds per suite (JVM)	Seconds per suite (simulator)
CI Runner OS & Cost	Linux (ubuntu-latest) Low cost	macOS (macos-14) Higher runner cost
UI Framework Coverage	Jetpack Compose & Classic Android Views	SwiftUI & UIKit (UIViewController)
Anti-Flakiness Guard	Fixed data, theme and device config; no system UI	Fixed Simulator version + tolerance (precision: 0.98)
CLI Tasks (Record / Verify)	`./gradlew record / verifyPaparazziDebug`	`isRecording = true / xcodebuild test`

Strategic Takeaway: Android achieves zero-infrastructure testing via JVM bytecode, while iOS utilizes off-screen native rendering with strict OS-image pinning.

PARALLEL PR RUNNERS

04D. IOS SNAPSHOT EXECUTION PROFILE

iOS Testing Engine: Advantages & Architectural Trade-offs

Core Advantages

100% Native Rendering: Runs inside a headless iOS Simulator process, ensuring authentic Apple San Francisco font typography and shadow rendering.

Unified UI Support: Single API (`assertSnapshot`) works across SwiftUI views and legacy UIKit ViewControllers.

Off-Screen Speed: Bypasses slow UI automation layer (XCUITest), rendering snapshots in seconds per suite.

Constraints & Pitfalls

macOS Runner Cost: Apple licensing requires macOS runners (macos-14), which are 5–10x more expensive per CI minute than Linux.

OS/Simulator Drift: Differences in minor iOS versions (e.g., 17.2 vs 17.4) cause font anti-aliasing diffs.

Mitigation Strategy: Enforce strict CI simulator version pinning and apply a tolerance factor (`precision: 0.98`).

Tool: Point-Free swift-snapshot-testing **Mitigation:** Precision tolerance & Runner pinning

Paparazzi on JVM & GitHub Actions Pipeline (implemented for Android)

Android (Paparazzi) Strengths

- **Zero Emulator Needed:** Uses Android Studio's Layoutlib to parse Jetpack Compose bytecode straight on host JVM.
- **Fast & Cheap:** Runs on standard Linux runners (ubuntu-latest); 24 snapshots plus build in about 1–3 minutes.
- **Deterministic:** System animations, battery bars, and network latency are completely bypassed.
- **Diff Reports:** Failed snapshots upload delta images (golden | diff | actual) and an Allure report with all three attached to each failed test.

```
./gradlew :app:verifyPaparazziDebug
```
Every PR is checked against approved goldens before merge.

scheduled_tests.yml: 3 jobs — GitHub Actions

JOB 1: lint: `spotlessCheck` (ktlint + license headers)

JOB 2: verify: `./gradlew :app:verifyPaparazziDebug` · Allure report · upload delta images & reports (14 days) · optional Telegram alert on failed nightly / main runs

JOB 3: record (manual only): records goldens on the same Linux runner and pushes them to a `goldens/review-<run>` branch for human review. Nothing is auto-approved.

Runs on every PR, on push to main, nightly at 02:00 UTC, and manually.

* iOS job (Swift-Snapshot-Testing on macOS) is part of the target architecture, not implemented in this take-home.

Why Appium Fails for Visual Regression

Top 3 Critical Issues

- **1. Extremely Slow Execution:** Commands travel over HTTP WebDriver, taking 30s+ per screen.
- **2. High Flakiness:** System animations, battery status bars, and network latency cause false diffs.
- **3. No Component Isolation:** Cannot snapshot isolated Jetpack Compose or SwiftUI views.

CI Cost & Maintenance Impact

- **Infrastructure Costs:** Heavy macOS/Android emulator VMs required in CI.
- **Mock Injection Complexity:** Requires local proxy servers (WireMock) to stub backend data.
- **Maintenance Overhead:** Constant locators updating when UI IDs change.

06. CI/CD INTEGRATION

Multi-Tiered CI/CD Execution Pipeline

TIER 1: EVERY PR

Fast Feedback · Duration: 3–5 mins

- Unit Tests
- Snapshot Tests (Paparazzi / Swift-Snapshot)
- Static Analysis (ktlint / SwiftLint)
- **Failed snapshots listed in the job summary, delta images in artifacts**

Gatekeeper: Blocks PR merge on visual regression

TIER 2: NIGHTLY BUILD

Full Regression · Duration: 45–60 mins

- Full E2E Flows
- Synthetic Load Testing
- Pact Contract Tests
- Memory Leak Scanning
- **Telegram alert when the nightly run fails**

Nightly stability & backend contract check

TIER 3: PRE-RELEASE

Production Audit · Per Release Cycle

- Real Device Cloud Matrix
- Accessibility Audits
- Bundle Size Regression
- Final Release Sign-off

Final production readiness gate

Load & Stress Testing Strategy

Backend & Push Storm Load

Simulate 50,000+ RPS breaking news push notifications using k6 / JMeter.

- **Target:** API Response Time p95 < 200ms
- **CDN Hit Ratio:** > 95% cache effectiveness

Client-Side Stress & FPS

Infinite scroll performance testing via Android Profiler & Xcode Instruments.

- **FPS Target:** > 58 FPS continuous rendering
- **Memory Leak Guard:** Zero OOM errors on 500+ items

Client-Side Stress Execution & Testing Cadence

Client Tools & Key Metrics

Android: Profiler, Perfetto · **iOS:** Xcode Instruments · **Cross:** Emerge, Firebase

Automated Stress Scenario:
500-item infinite scroll (100 cards/min) + Heavy Rich Media (GIFs, Tweets, SVG, Video).

- **FPS Target:** 55–60 FPS (Jank frames < 2%)
- **CPU Usage:** Zero thermal throttling under load
- **RAM Footprint:** Flat memory line (No OOM leaks)

Stage / Event	Feasibility	Testing Scope
Every Sprint / CI	Not Feasible	App size regression profiling only
Major Events	**Critical**	Backend Push Storm (Simulated Peak)
New UI Engine	**Mandatory**	Client Stress (Scroll & RAM on low-end)
Quarterly Run	Recommended	Infra capacity & Auto-scaling limits

Scrum Rollout & Team Mentorship

1

Pilot Phase

Integrate Paparazzi / Swift-Snapshot into 1 core feature module (e.g., Article Teasers).

2

Dev Enablement & Mentoring

Train Android & iOS engineers alongside mentoring junior QA to author snapshots and review diffs confidently.

3

CI Pipeline Gate

Enforce snapshot checks on PRs with automated diff previews in Danger/GitHub comments and Telegram alerts.

AI Prompt Engineering Lifecycle

Phase 1: Architecture & Rationale
Formulating toolchain trade-offs, off-device execution, and mock fixtures.

Phase 2: Domain Stress & Edge Cases
Scaffolding long German headlines, paywalls, RTL, and tablet configurations.

AXEL SPRINGER AI VISION

The C-R-E-A-T-E Framework

LLMs and an AI coding agent drafted the strategy, tests and CI setup. Every change went through a pull request with green CI, and every golden was checked against a review checklist before approval. Prompts and lessons learned: docs/ai-usage.md

Phase 3: CI/CD & Notification Bot
Automating GitHub Actions workflows, artifact retention, and Telegram alerts.

What Was Built: JetNews Visual Regression Suite

The Suite

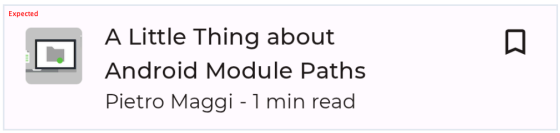
- **24 Paparazzi tests** on the app's real composables: `PostCardTop`, `PostCardSimple`, `PostCardHistory`, `PostCardPopular`, `HomeFeedScreen`, `PostScreen`, plus a `BreakingNewsBanner` and a Premium badge added to the app
- Light & dark theme, long German compound headlines, 150% & 200% font scale, Arabic RTL, tablet wide-window, empty state
- Components cropped to their size (`SHRINK`); data from the app's own fixed posts, no network
- Severity lives only in Allure annotations; failed tests get golden, actual and delta attached
- Written strategy: `STRATEGY.md`

Real Composable Coverage · Deterministic Fixtures

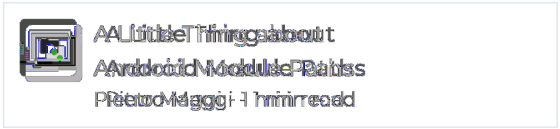
A Real Regression (PR #2)

• **One line changed in the app:** thumbnail padding 16 dp → 12 dp
• **Result:** 11 of 24 tests failed, exactly the ones that show this card; the other 13 passed

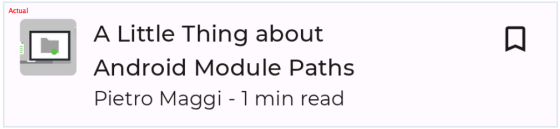
Golden (approved): thumbnail padding 16 dp



Difference: the content moved, so the text is doubled



Actual (this PR): thumbnail padding 12 dp



Real Paparazzi delta image from the failed CI run of PR #2 (TR-04)

Asset: `docs/regression-demo/delta-simpleCard_default.png`

Golden Review Found 5 Issues

- **Test defect (fixed before approval):** Popular card stretched to full screen height. Its weighted Spacer filled all available height; the test now hosts it in a Row with IntrinsicSize.Max, as the app does.
- **Product issue:** German words break mid-word without hyphenation on narrow phone screens.
- **Product issue:** «min read» is not translated to Arabic, so the line mixes Arabic and English and reads as «min read 1 - <author>».
- **Test setup issue (fixed):** dynamic color showed the renderer's palette, not the brand colors. Tests now render the brand schemes.
- **Product defect in the new banner (fixed before approval):** at 200% font the update time broke mid-word.

Human-in-the-loop Baseline Validation

A golden image is only as good as the review it got.

AXEL SPRINGER TECH QUALITY ASSURANCE

Thank You! Q&A

Ready to walk through the runnable demo & code implementation.

PRESENTER
Ilona Zakharava

CODE REPOSITORY
github.com/ilonazakharova/jetnews-visualregression